

FACULTY OF SCIENCE AND COMPUTING

DEPARTMENT OF COMPUTING

COURSE NAME: Object Oriented C++

COURSE CODE: BSE 2102

LECTURER: Mr. Mwanje Abiyasaali

CAMPUS: Main

2026/2027 ACADEMIC YEAR

AUGUST –DECEMBER

Name	Registration Number	Role in Project
MUGUME JULIUS	25/1/370/D/1497	savings and loan modules
KASIRYE CHRISTOPHER	25/1/370/D/2175	File handling and testing

Ndejje Farmers SACCO Loan Management Information System.

Abstract.

Ndejje Farmers SACCO keeps member, savings and loan records in paper books which makes interest calculations slow, balances hard to verify and records easy to lose. This project developed a menu driven C++ Loan Management Information System that registers farmers, tracks savings and share capital, checks loan eligibility, calculates interest and monthly installments, records repayments and produces management reports. All data is saved in text files, so records remain after the program closes. The system uses classes, inheritance, vectors, file streams and strict input validation. It was compiled with two different compilers and passed all 64 test cases.

1. Introduction.

1.1 Background

Savings and Credit Cooperative Organisations (SACCOs) are the main source of affordable credit for farmers in rural Uganda. Members pool their savings and share capital and the SACCO lends that money back to members at interest. Registered SACCOs grew and were driven largely by government programmes such as the Parish Development Model and Emyooga .

Ndejje lies in Luweero District and it became a town council in July 2020. Most households around the town council depend on small scale farming so a farmer owned SACCO is a key source of money for seeds, fertiliser, livestock and small farm businesses.

The case study for this project is Ndejje Farmers SACCO, a farmer owned cooperative that collects savings and share capital from its members and lends to them. SACCOs are Tier four institutions supervised by the Uganda Microfinance Regulatory Authority (UMRA). They must send quarterly financial reports to UMRA and falsifying their records is an offence. Accurate, well kept records are therefore a legal duty not just good practice.

1.2 Problem Statement.

Ndejje Farmers SACCO records members, savings, loans and repayments by hand in paper registers and ledger books. This causes five problems;

1. Slow service. Finding one member's balance means searching through several books so farmers queue for a long time.
2. Calculation errors. Interest and monthly installments are worked out by hand or on a phone calculator, so two officers can get different answers for the same loan.
3. Weak loan control. There is no quick way to check whether a member already has an unpaid loan, has enough savings or is behind on payments.
4. Risk of loss. Paper books can be torn, lost, burnt or altered and there is no backup.
5. Slow reporting. Totals for savings, loans out and money owed must be added up by hand before every management meeting or audit.

These problems delay loans, cause disputes with members and make the SACCO hard to audit.

1.3 Objectives.

Main objective is to develop a menu driven C++ Loan Management Information System that automates member, savings and loan records for Ndejje Farmers SACCO and stores them permanently in files.

Specific objectives.

1. To register members and allow staff to view, search, update and delete member records.
2. To record savings deposits, withdrawals and share capital contributions in a ledger with running balances.
3. To capture loan applications and check them automatically against the SACCO's eligibility rules.
4. To manage each loan through the stages Pending, Approved, Active and Completed or rejected.
5. To calculate interest and a monthly repayment schedule using the reducing balance method.
6. To record repayments, update balances automatically and flag loans that are behind schedule.
7. To produce reports on total savings, active loans, arrears, daily transactions and individual member statements.

8. To store all records in files so that no data is lost when the program closes.

1.4 Scope.

Dimension	Covered	Not covered
Geographical	Ndejje Farmers SACCO office, Ndejje Town Council, Luweero District	Other branches or other SACCOs
Functional	Members, savings, share capital, loans, repayments, search, reports, staff logins, backups and audit log	Mobile money, dividends, full accounting and SMS alerts
Users	Administrator (manager), Loan Officer, SACCO members (read only self service)	Online access by the public
Technical	Single computer console program in standard C++ with text file storage	Networked database, graphical interface
Currency	Uganda Shillings (UGX), whole shillings	Foreign currency

1.5 Significance.

- SACCO managers get instant totals of savings, loans and arrears, which supports better lending decisions and faster reports to UMRA.
- Loan officers no longer calculate interest by hand. The system applies the same rules and formula to every loan.
- Farmers (members) get faster service, printed-style statements and a clear repayment schedule showing exactly what they owe each month.
- Auditors and the board can trace every transaction to the staff member who recorded it, the date and the time, using the transaction ledger and audit log.
- Students and researchers get a working example of how C++ object-oriented programming and file handling solve a real community problem.

2. System Analysis and Design.

2.1 Existing System and Its Problems.

Today every record at Ndejje Farmers SACCO is written by hand. A new member fills a paper form and is entered in a member register. Savings and share payments are written in the member's passbook and in a cash book. A loan applicant fills a paper form, the loans committee decides at a meeting, and the loan officer works out the interest on a calculator and writes the installments on a loan card. At month end the treasurer adds up every book by hand.

Activity	How it is done now	Problem it causes
Member records	Paper register, filed forms	Duplicate members; slow look ups; details lost when pages tear
Savings and shares	Passbook and cash book	Passbook and cash book often disagree; no running balance
Loan eligibility	Officer checks books by memory and judgement	Rules applied unevenly; members with unpaid loans can borrow again
Interest and installments	Hand or phone calculator	Arithmetic errors; members dispute amounts
Repayments and arrears	Loan cards updated by hand	Late payers noticed only weeks later
Reports	Totals added by hand at month end	Days of work; figures hard to check
Security and audit	Books kept in a cupboard	No record of who changed what; no backup

2.2 Proposed System.

The proposed system is a menu driven C++ console program that runs on one computer in the SACCO office. Staff log in with a username and password; members can view their own account by entering their Member ID and National ID Number (NIN). Every action is chosen from a numbered menu and every value typed is checked before it is accepted.

The program has six modules.

1. Member management – register, view, update and delete members.
2. Savings and share capital – deposits, withdrawals, share purchases and a ledger per member.
3. Loan management – eligibility check, application, approval or rejection, payout, repayment and a loan calculator.
4. Search – by Member ID, NIN, name (full or part), phone number or Loan ID.

5. Reports – financial summary, member register, active loans, arrears, member statements and daily transactions.
6. Administration – loan policy, staff accounts, backups, audit log and demonstration data.

All records are kept in memory in vectors while the program runs and are saved to six text files after every change, so nothing is lost when the program closes.

2.3 Functional Requirements.

The system shall;

- FR1. Register a member with name, NIN, phone, address, date joined, shares and opening savings and give an automatic Member ID (NFS0001, NFS0002 …).
- FR2. Refuse a NIN that is already registered.
- FR3. List all members, show one member's details, update personal details and delete a member who has no unfinished loan.
- FR4. Record savings deposits, withdrawals and share purchases each as a ledger entry with the balance after it.
- FR5. Block withdrawals while the member has an unfinished loan and keep a minimum balance of UGX 10,000.
- FR6. Check a loan request against seven eligibility rules and show each rule as PASS or FAIL.
- FR7. Save an eligible application as PENDING and allow it to be updated or cancelled before payout.
- FR8. Let only an Administrator approve or reject a PENDING loan.
- FR9. Pay out an APPROVED loan, set its due dates and mark it ACTIVE.
- FR10. Calculate the monthly installment, total interest and full repayment schedule using the reducing-balance method.
- FR11. Record repayments, reduce the balance, refuse overpayment and mark the loan COMPLETED when fully paid.
- FR12. Work out arrears, days overdue and a risk category for every active loan.
- FR13. Search members by Member ID, NIN, name or phone, and loans by Loan ID.
- FR14. Produce the financial summary, member register, active loans, arrears, member statement and daily transactions reports, and export a statement to a text file.
- FR15. Save all data to files after every change and load it again when the program starts.
- FR16. Require a staff login, support Administrator and Loan Officer roles, and record important actions in an audit log.

2.4 Non functional Requirements.

Quality	Requirement	How the system meets it
Reliability	No crash on bad input or damaged files	Whole line input with validation; damaged file lines skipped and a copy kept; exceptions caught
Data safety	No data loss on power failure	Saved after every change; new file written to a temporary copy first, then swapped in
Usability	Usable by staff with basic computer skills	Numbered menus, plain-English messages, confirmation before every save, Enter keeps current values
Performance	Instant response for a large SACCO	10,000 members, 30,000 transactions and 3,000 loans loaded, searched, reported and saved in about 1 second in testing
Security	Only authorised staff change data	Login with masked password, stored only as a hash; Administrator only functions; 3 failed logins block access
Auditability	Every money movement traceable	Ledger entries are never edited and record the staff username; audit log of actions
Portability	Runs on common PCs	Standard C++; compiles cleanly from C++98 to C++20 on g++ and clang++; Windows and Linux code paths
Maintainability	Easy to change rules	Loan policy stored in a settings file and editable from the menu; code split into small classes and files

2.5 System Flowchart.

The main algorithm of the program is;

1. Start. Load settings, members, loans, transactions and staff users from the data files. If no staff file exists, create the default logins.
2. Show the welcome menu: 1 Staff Login, 2 Member Self-Service, 0 Exit.
3. If Staff Login: read username and password. If wrong, allow up to 3 attempts, then return to step 2.

4. Show the main menu and read a choice. If the choice is not 0–7, show an error and ask again.
5. Run the chosen module (members, savings, loans, search, reports, administration or change password).
6. For any change: validate the input, show a summary, ask for confirmation, update the records, write a ledger entry if money moved, save all files and write to the audit log.
7. Return to the menu. Repeat steps 4–6 until the user chooses Logout, then go back to step 2.
8. If Member Self-Service: read Member ID and NIN. If they match, show the read-only member menu; otherwise deny access.
9. If Exit: save all files and stop.

Loan processing inside step 5 follows this path: check eligibility → if any rule fails, refuse; otherwise save as PENDING → Administrator approves (APPROVED) or rejects (REJECTED) → payout sets due dates (ACTIVE) → repayments reduce the balance → balance reaches zero (COMPLETED).

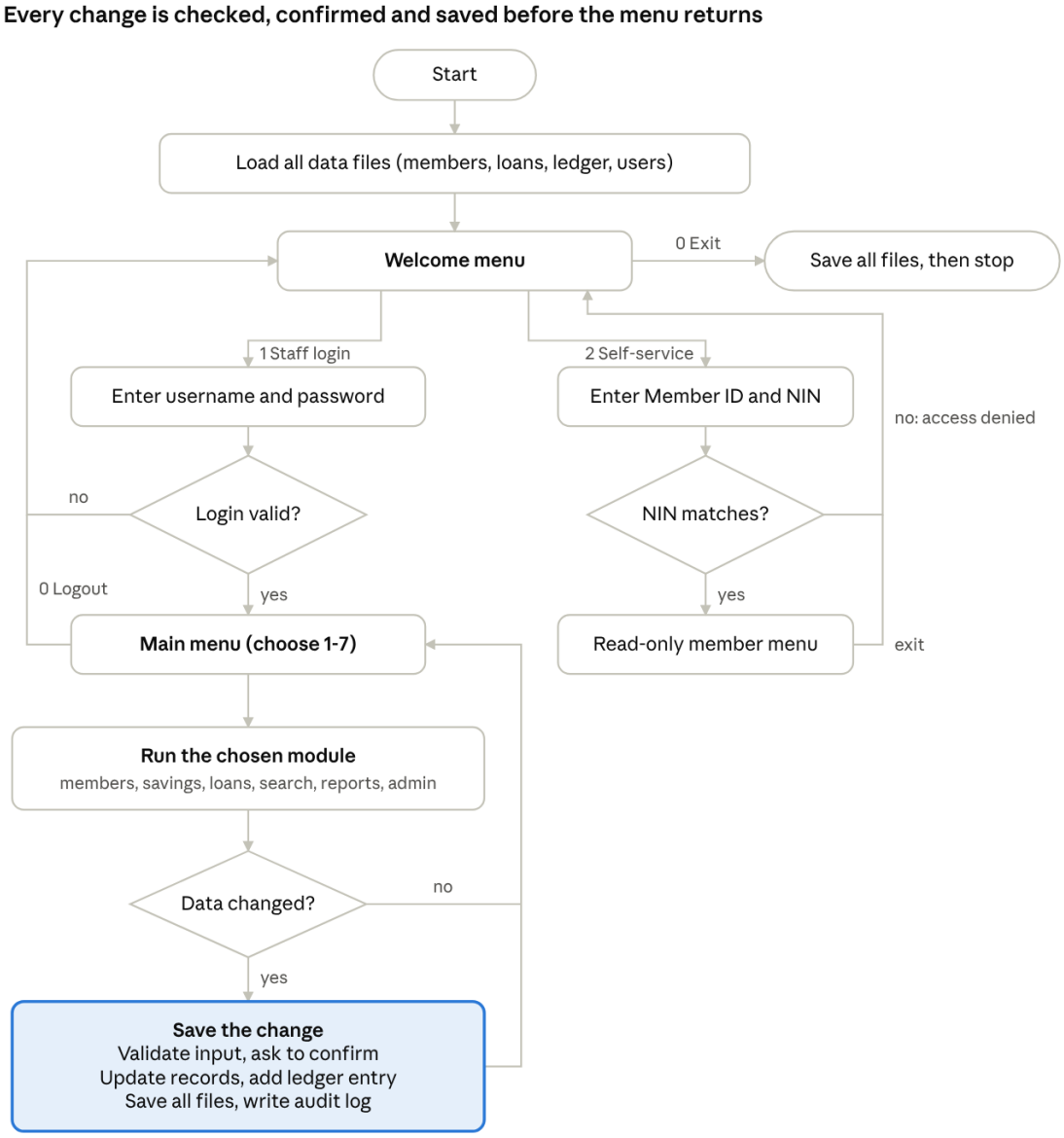


Figure 2.1. System flowchart.

Shows the program flow: wrong logins and a failed NIN check return to the welcome menu, and every data change passes the save step before the main menu returns.

2.6 Use Case Diagram.

The system has three actors. The Administrator can do everything a Loan Officer can do, plus the restricted tasks.

Actor	Use cases
SACCO Member	View account summary; view statement; view loans and repayment schedule; check how much they can borrow

Actor	Use cases
	(self-service, read-only). Deposits, withdrawals and loan applications are done for the member by a Loan Officer.
Loan Officer	Log in and change password; register, view and update members; search records; deposit and withdraw savings; buy shares; view ledger; check eligibility; apply for a loan; update or cancel an application; pay out an approved loan; record repayments; use the loan calculator; view and export reports.
Administrator	All Loan Officer use cases, plus: approve or reject loans; delete members; update loan policy; add, reset or deactivate staff users; back up data; view audit log; load demonstration data.

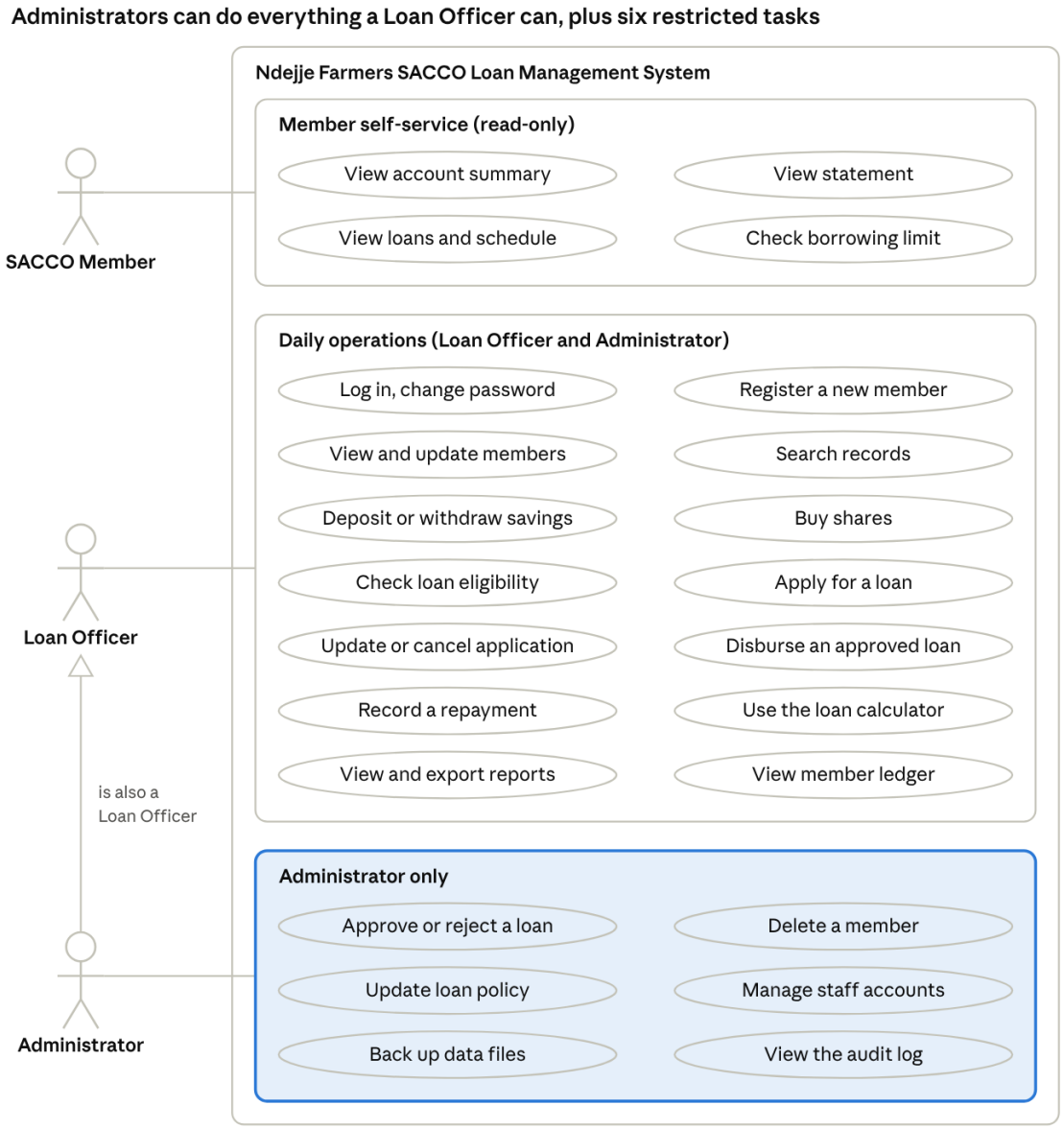


Figure 2.2. Use Case Diagram.

The hollow arrow means the Administrator inherits every Loan Officer use case; the shaded group holds the six tasks only an Administrator may perform.

3. System Implementation

3.1 Development Environment.

Item	Details
Language	Standard C++ (written to the C++11 standard; also compiles as C++98, 14, 17 and 20)
Compilers used for testing	Microsoft Visual C++ in Visual Studio (Release build, /MT) for the submitted .exe; g++ 13.3.0 and clang++ 18.1.3 for the automated test runs
Operating system used for testing	Windows (development in Visual Studio, manual testing and screenshots by the group); Ubuntu 24.04 LTS
Target platforms	Windows 10/11 (Code::Blocks or Dev-C++ with MinGW g++, or Visual Studio 2019/2022) and Linux

Item	Details
Editor	Visual Studio
Build command	`g++ -std=c++11 -Wall -O2 -o NdejjeSACCO *.cpp`
Size	25 source files (11 headers, 14 .cpp files)

3.2 C++ Concepts Used.

Required concept	Where it is used
Variables and data types	`std::string` for IDs, names and dates; `double` for money and rates; `int` for months and counters; `bool` for flags; `unsigned long long` for the password hash; `enum` for loan status, transaction type and user role
Input and output	`std::getline(std::cin, …)` reads whole lines; `std::cout` with `<iomanip>` (`setw`, `left`, `right`) prints aligned tables
if, if…else, nested conditions	Eligibility rules, login checks, withdrawal checks (open loan → minimum balance → amount) in `SaccoMembers.cpp`
switch	Every menu; loan purpose selection; status and type names in `Loan.cpp` and `Transaction.cpp`
for, while, do…while	`for` over vectors in reports and searches; `while (true)` input loops that repeat until input is valid; `do…while` for every menu
Functions	More than 200 small, named functions; `main()` only creates the system object and calls `run()`
Arrays	`statusCount[5]` and `typeTotals[6]` in reports; arrays of demonstration records
Structures	`Installment` (one schedule row), `EligibilityRule`, `EligibilityResult`
Classes and objects	`Person`, `Member`, `StaffUser`, `Loan`, `Transaction`, `Settings`, `FileManager`, `SaccoSystem`
Constructors	Default and parameterised constructors in every entity class, using initialiser lists
Encapsulation and access specifiers	Balances are `private` and change only through `depositSavings()`, `withdrawSavings()` and `addShareCapital()`, which refuse negative or excessive amounts; `protected` fields in `Person`; `public` interfaces
Inheritance and polymorphism	`Member` and `StaffUser` inherit from the abstract class `Person` and override the pure virtual functions `getRoleTitle()` and `displayProfile()`
Vectors and other containers	`std::vector<Member>`, `std::vector<Loan>`, `std::vector<Transaction>`, `std::vector<StaffUser>`; `std::deque` for reading the last lines of the audit log
File handling	`std::ifstream`, `std::ofstream`, append mode for the audit log, binary copy for backups
Searching	Linear search by ID, NIN, phone and case-insensitive part of a name
Validation	`InputHelper` validators for names, NIN, phone, dates, money, menu choices
Error handling	`try/catch` for damaged data (`std::invalid_argument`), a custom `InputClosedException`, and a safety net in the main menu

3.3 System Architecture.

The program is built in four layers. `main()` creates one `SaccoSystem` object, the controller. The controller holds every record in vectors, applies the business rules and draws the menus. It uses the entity classes for the data, `InputHelper` for safe input, `Utils` for formatting and dates, and `FileManager` for all reading and writing of files.

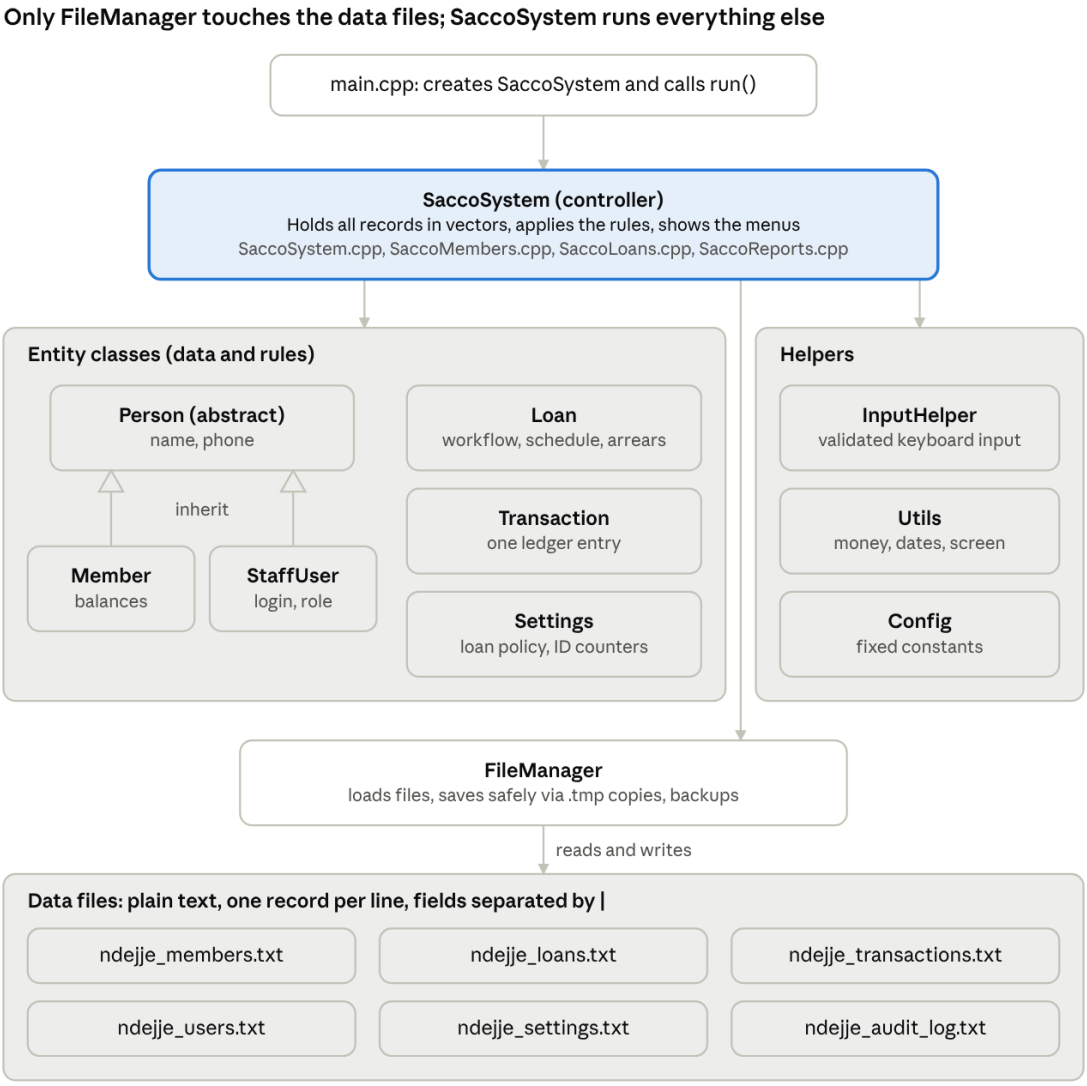


Figure 3.1: System architecture

Shows the layers: the controller uses the entity and helper classes, and only FileManager reads or writes the six data files.

File(s)	Contents
`main.cpp`	Program entry point
`SaccoSystem.h` + 4 .cpp files	Controller: `SaccoSystem.cpp` (start-up, saving, login, menus, administration), `SaccoMembers.cpp` (members, savings, shares), `SaccoLoans.cpp` (eligibility and loan workflow), `SaccoReports.cpp` (search, reports, self-service)
`Person.h/.cpp`	Abstract base class for people
`Member.h/.cpp`, `StaffUser.h/.cpp`	Member and staff classes derived from `Person`
`Loan.h/.cpp`	Loan workflow, amortisation schedule, repayments, arrears
`Transaction.h/.cpp`	Ledger entries
`Settings.h/.cpp`	Loan policy and ID counters
`FileManager.h/.cpp`	File loading, safe saving, backups
`InputHelper.h/.cpp`	Validated keyboard input
`Utils.h/.cpp`, `Config.h`	Helpers (text, money, dates, screen) and fixed constants

3.4 Data File Structure.

The system acts as its own small database. Each table is a plain text file in the program folder with one record per line and fields separated by the `|` symbol. The first line of each file is a `#` comment naming the columns, so the files can be read by a person or opened in a spreadsheet.

File	Fields (in order)
`ndejje_members.txt`	MemberID, FullName, NIN, Phone, Address, DateJoined, ShareCapital, SavingsBalance

File	Fields (in order)
ndejje_loans.txt	LoanID, MemberID, Principal, InterestRate, Months, Purpose, Status, AppliedOn, DecisionDate, DisbursedOn, AmountPaid, AppliedBy, DecidedBy, Remarks
ndejje_transactions.txt	TxnID, MemberID, Type, Amount, BalanceAfter, Date, Time, Description, LoanID, RecordedBy
ndejje_users.txt	Username, PasswordHash, Role, FullName, Phone, Active
ndejje_settings.txt	key=value lines: interest rate, loan multiplier, limits, ID counters
ndejje_audit_log.txt	Date and time, user, action (one line per action, append only)

3.5 Important Implementation Decisions.

1. Reducing-balance interest. The installment uses the amortisation formula below, with r = annual rate ÷ 12 ÷ 100 and n = months. Each month's interest is charged only on the balance still owed. Amounts are rounded to whole shillings, and the last installment absorbs the rounding so the balance ends at exactly zero.

```
Installment = P × r × (1 + r)^n ÷ ((1 + r)^n − 1)
```

2. Vectors instead of fixed arrays. The number of members is not limited, and vectors manage their own memory, so the program uses no `new` or `delete`. A memory-checking build (AddressSanitizer and UndefinedBehaviourSanitizer) ran all 64 tests with zero errors.
3. Safe saving. Each file is written to a `.tmp` copy, then swapped in. If the power fails mid-save, the old file or the `.tmp` copy survives, and the loader uses the `.tmp` copy when the main file is missing.
4. Save after every change. The files are small, so all of them are rewritten after each change. This keeps them consistent with each other.
5. Whole-line input. Input is read as a full line and then checked, which prevents the infinite loop that `cin >> number` causes when letters are typed.
6. Ledger entries are never edited. Mistakes are corrected with a new entry and each entry stores the staff username, as auditors expect.
7. IDs are never reused. Counters in the settings file only go up, so a deleted member's ID cannot be given to someone else.
8. Each loan keeps its own interest rate. Changing the policy affects new loans only.
9. Passwords are hashed. Only a 16 character hash is stored; the password is masked with * as it is typed.
10. One function, two outputs. Statements are written to a `std::ostream&`, so the same code prints to the screen or to a text file.
11. Portability. Number conversion is written by hand (instead of `std::stod`, which some older MinGW compilers lack) and password masking uses `#ifdef _WIN32` to pick the Windows or Linux method.

4. Testing.

All 64 test cases passed, with no crashes on three separate builds of the program.

4.1 Test Approach.

- Method: black box functional testing. Each test starts the compiled program from a known data set: an empty folder or the demonstration data loaded through Administration > Load Demonstration Data (6 members, 5 loans, 33 transactions). The typed keystrokes come from a prepared input file and the program's real screen output and data files are checked.
- Calculations checked independently: every interest, installment and balance figure was re calculated separately with the amortisation formula before being accepted.
- Password masking was checked on a real terminal: the password showed as `********`.
- Manual testing on Windows. We compiled and ran the program in Visual Studio and took screenshots of the main features (see the separate Testing Evidence document).

4.2 Test Results.

Test Case	Input	Expected Result	Actual Result	Status
T01 Add member record	Valid member details (Ssali Moses, CM93012345JKLM, 0772987654, 5 shares, UGX 100,000)	Member registered	Member registered as NFS0007 and saved to file	Pass
T02 Name validation	'John123', then 'Moses', then valid name	Errors shown, re-prompted	Both errors shown; name asked again each time	Pass
T03 NIN validation	NIN '12345'	Error message, re-prompted	NIN format error shown; asked again	Pass
T04 Duplicate NIN	CM78001234ABCD (used by NFS0001)	Error, member not	"Already registered to NFS0001";	Pass

Test Case	Input	Expected Result	Actual Result	Status
		saved	nothing saved	
T05 Phone validation	'12345', then '+256 772 987 654'	Error, then stored as 0772987654	Error shown; stored as 0772987654	Pass
T06 Date validation	2030-01-01, then 13-10-2026	Errors shown, re-prompted	Both date errors shown; asked again	Pass
T07 View all members	Member menu option 2	Table with totals	6 members listed with totals	Pass
T08 View member details	Member ID typed as '1'	Profile of NFS0001	NFS0001 profile and loan LN0001 shown	Pass
T09 Update member record	NFS0006, new phone 0700111222	Record updated	Updated and saved to file	Pass
T10 Update with duplicate NIN	NFS0006 given NFS0001's NIN	Error, no change	"NIN already belongs to NFS0001"; nothing saved	Pass
T11 Delete record	Valid Member ID NFS0006	Record deleted	Record deleted; UGX 70,000 refund recorded	Pass
T12 Delete member with loan	NFS0001 (active loan)	Delete refused	Refused: unfinished loan LN0001	Pass
T13 Delete by Loan Officer	Officer login, delete option	Access denied	"Access denied. Only an Administrator…"	Pass
T14 Search record	Valid Member ID NFS0003	Record displayed	Record displayed (Kizza Richard)	Pass
T15 Search record	Invalid Member ID NFS0999	Error message displayed	Error message displayed	Pass
T16 Search by NIN	cf85023456bcde (lower case)	Record displayed	Record displayed (Nakato Florence)	Pass
T17 Search by NIN	Unregistered NIN	Error message	"No member found with NIN…"	Pass
T18 Search by part of name	'na'	All matches listed	3 records listed	Pass
T19 Search by name	'Zzz'	Error message	"No member found whose name contains 'Zzz'"	Pass
T20 Search by phone	+256756345678	Matching member	1 record listed (Kizza Richard)	Pass
T21 Deposit savings	NFS0006, UGX 50,000	Balance 30,000 → 80,000	New balance UGX 80,000	Pass
T22 Input validation	Deposit '-5000', 'abc', '2.5'	Each rejected, re-prompted	Three error messages; then 50,000 accepted	Pass
T23 Withdraw with active loan	NFS0001	Withdrawal blocked	Blocked: savings secure loan LN0001	Pass
T24 Withdraw too much	NFS0003: 300,000, then 100,000	First rejected, second accepted	Max UGX 240,000 shown; balance UGX 150,000	Pass
T25 Buy shares	NFS0006, 3 shares	Share capital UGX 100,000	UGX 100,000 (5 shares)	Pass
T26 Member ledger	NFS0002	All entries with balances	5 entries incl. payout and repayment	Pass
T27 Eligibility (eligible)	NFS0003, UGX 500,000, 6 months	All rules pass, cost shown	7 of 7 PASS; installment UGX 87,763	Pass
T28 Eligibility (not eligible)	NFS0006, UGX 200,000	Failed rules listed	NOT ELIGIBLE: 3 rules failed	Pass
T29 Input validation	Negative loan amount (-500000), then 500000	Error handled/re-prompted	Error handled/re-prompted; LN0006 saved as PENDING	Pass
T30 Above borrowing limit	NFS0003 asks UGX 5,000,000	Refused	Rule 6 FAIL; not registered	Pass
T31 Period too long	36 months	Refused	Rule 7 FAIL; not registered	Pass
T32 One loan at a time	NFS0001 applies again	Refused	Refused: LN0001 still active	Pass
T33 Approve loan	Admin approves LN0004	PENDING → APPROVED	APPROVED and saved	Pass
T34 Reject loan	LN0004 with reason	PENDING → REJECTED	REJECTED with reason saved	Pass
T35 Approve by Loan Officer	Officer, approve option	Access denied	Access denied	Pass
T36 Disburse approved loan	LN0005	APPROVED → ACTIVE, due dates set	ACTIVE; first installment due one month after payout	Pass
T37 Disburse pending loan	LN0004	Refused	"Loan LN0004 is PENDING"	Pass

Test Case	Input	Expected Result	Actual Result	Status
T38 Record repayment	LN0002, UGX 263,288	Balance 1,053,150	Balance UGX 1,053,150; overdue UGX 526,576	Pass
T39 Overpayment	LN0001: 2,000,000, then 183,360	First rejected	Max UGX 1,283,521 shown; balance UGX 1,100,161	Pass
T40 Final repayment	LN0001 full balance	ACTIVE → COMPLETED	Loan FULLY PAID (COMPLETED)	Pass
T41 Repay completed loan	LN0003	Refused	Refused: LN0003 is COMPLETED	Pass
T42 Update application	LN0004 amount 900,000	Updated after re-check	Updated to UGX 900,000	Pass
T43 Cancel application	LN0004	Deleted	Cancelled and deleted	Pass
T44 Interest calculation	UGX 1,000,000, 12 months, 18%	Installment 91,680; interest 100,161	Installment UGX 91,680; interest UGX 100,161	Pass
T45 Loan schedule	LN0002	PAID and OVERDUE rows	PAID, OVERDUE, UPCOMING rows; 61 days late	Pass
T46 Financial summary	Reports option 1	Correct totals	Savings UGX 1,760,000; PAR 50.6%	Pass
T47 Active loans report	Reports option 3	Active loans with totals	2 loans with totals	Pass
T48 Arrears report	Reports option 4	Late loans listed	LN0002: 61 days, Substandard	Pass
T49 Member statement	NFS0002	Full statement	Transactions, loans and net position shown	Pass
T50 Daily transactions	Today	Day's totals	Entries listed; net cash UGX 198,360	Pass
T51 Export statement	NFS0002	Text file created	statement_NFS0002_<date>.txt created	Pass
T52 Invalid menu choice	'9', then 'abc'	Error, re-prompted	"Invalid choice…" shown each time	Pass
T53 Wrong password	3 wrong attempts	Login blocked	Blocked after third attempt	Pass
T54 Change password	New password Ndejje2026	New password works	Changed; login with new password succeeded	Pass
T55 Member self-service	NFS0002 + correct NIN	Read-only summary	Summary shown (savings UGX 400,000)	Pass
T56 Self-service wrong NIN	NFS0002 + wrong NIN	Access denied	Access denied	Pass
T57 Change interest rate	Policy rate 20%	New loans use 20%	Saved; calculator offers 20.00%	Pass
T58 Add staff user	New officer 'jnamata'	User can log in	Created; "Welcome, Namata Joan!"	Pass
T59 Back up data	Administration option 7	Backup copies made	6 files backed up	Pass
T60 Audit log	Administration option 8	Latest actions listed	Entries with date, time and user	Pass
T61 Data persists	Register, exit, restart, search	Record still there	Record found after restart	Pass
T62 First run	Empty folder	Default logins, empty system	Logins created; "No members registered yet"	Pass
T63 Damaged data line	Corrupt line in members file	Skipped with warning	Warning shown; 6 good members loaded	Pass
T64 Input ends suddenly	Input closed during a deposit	Clean exit, no crash	"Input closed. All data has been saved."	Pass

4.3 Sample Output Evidence

The full screen transcripts and input files for all 64 tests are supplied in the `TestingEvidence` folder. Three short extracts follow.

Eligibility check that fails (T28):

```
  No     Rule                                                                                    Result     Details
  1       Member  for  at  least  3  month(s)                                          FAIL          Member  for  1  month(s)
  2         Share  capital  at  least  UGX  100,000                                        FAIL            Has  UGX  40,000
  3        Savings  at  least  UGX  50,000                                                   FAIL           Has  UGX  30,000
  4         No  other  unfinished  loan                                                              PASS             None
  5         Amount  between  100,000  and  20,000,000                                 PASS           Asked  for  UGX  200,000
  6          Amount  within  3.0  x  (savings  +  shares)                                 PASS            Limit  UGX  210,000
  7       Period  of  1  to  24  months                                                    PASS         Asked  for  6  month(s)
  RESULT: NOT ELIGIBLE - 3 rule(s) failed (marked FAIL).
```

Repayment on a late loan (T38):

```
  Monthly                installment                                    :                UGX                263,288
  Outstanding                balance                                   :                UGX              1,316,438
  Overdue            (arrears)                                                 :             UGX              789,864
  Repayment                              amount                                (UGX):                           263288
  [SUCCESS]          Repayment          recorded.          Outstanding          balance:          UGX          1,053,150
  [WARNING] Still overdue: UGX 526,576
```

Arrears report (T48):

```
  Loan  ID  Member                      Phone                      Overdue      Days    Category            Outstanding
  LN0002    Nakato  Florence        0701234567              789,864          61    Substandard            1,316,438
  Loans       in       arrears                                                                                :       1
  Portfolio at risk (PAR)   : 50.6% of all active loan balances
```

4.4 Defects Found and Fixed.

Defect	Fix
Usernames of different lengths made the audit log columns uneven	Added `Utils::padRight()` so every username fills 12 characters
`std::stod` and `std::stoi` are missing on some older MinGW compilers used on Windows	Wrote portable `numberOrThrow()` and `wholeNumberOrThrow()` functions
A lambda function stopped the code compiling under older C++ settings (Dev-C++ default)	Replaced it with a named comparison function; code now compiles from C++98 to C++20
The hand-worked expected interest for T44 (UGX 100,155) disagreed with the program (UGX 100,161)	An independent re-calculation confirmed UGX 100,161; the error was in the hand working, so the test plan was corrected

5. Conclusion.

The project met its general objective: Ndejje Farmers SACCO now has a working C++ system that replaces its paper registers for members, savings and loans and keeps every record permanently in files.

5.1 Achievements.

Specific objective	What was achieved
1. Member records	Register, view, search, update and delete members, with NIN and phone checks and duplicate NIN protection
2. Savings and shares	Deposits, withdrawals and share purchases, each recorded in a ledger with the balance after it
3. Eligibility	Seven configurable rules checked automatically and shown as a PASS/FAIL checklist
4. Loan workflow	Pending → Approved → Active → Completed (or Rejected), with Administrator-only approval
5. Interest	Reducing balance installments and full schedules, verified against independent calculation
6. Repayments	Balances update immediately; overpayment refused; arrears, days late and risk category shown
7. Reports	Financial summary, member register, active loans, arrears, statements (screen or file) and daily cash report
8. Storage	Six text files saved safely after every change; data survives restarts, damaged lines and sudden shutdowns

In addition, the system has staff logins with two roles, member self-service, an audit log, backups and demonstration data. It passed all 64 test cases on three builds.

5.2 Challenges.

- Reading numbers safely. `cin >> number` enters a failed state when letters are typed. The solution was to read every input as a full line and convert it afterwards.
- Aligning tables. Money with thousands separators has different widths, so each column needed fixed widths with `setw` and long names had to be shortened with "...".
- Different consoles. Clearing the screen and hiding passwords work differently on Windows and Linux, which required `#ifdef _WIN32` code for each.
- Saving files safely. Windows cannot rename a file over an existing one, so the safe-save routine removes the old file before renaming the new copy. Files edited on Windows also add `\r` to each line, which had to be stripped.
- Keeping files consistent. Members, loans and transactions are in separate files that must agree, so all files are saved together after every change.
- Money rounding. Rounding each installment to whole shillings left small differences which are absorbed by the last installment.
- Date arithmetic. Due dates at month ends and in leap years had to be worked out without external libraries.

5.3 Limitations.

- Only one person can use the data at a time on one computer since there is no network access.
- The data files are plain text and not encrypted, so anyone who can open the program folder can read them. Passwords are hashed but with a simple algorithm.
- It is a console program: no mouse, no graphics and no direct printing (statements are exported as text files to print).
- It does not handle guarantors, collateral, penalty interest on late loans, dividends or a full accounting general ledger.
- It has no link to mobile money or SMS so members still pay cash at the office.
- Searching checks every record one by one which is fast for tens of thousands of records but would slow down for millions.
- Dates come from the computer clock so a wrong clock gives wrong due dates.

5.4 Recommendations.

1. Move to a SQL database such as SQLite or MySQL so several staff can work at once and data can be encrypted and backed up automatically.
2. Build a graphical or web interface (for example with Qt, or a web front end) so staff can use a mouse, print directly and view charts.
3. Integrate mobile money (MTN MoMo and Airtel Money) for deposits and repayments and send SMS reminders before each due date.
4. Add guarantors, collateral, penalty interest and dividends to match the SACCO's full loan policy and by laws.
5. Strengthen security with a modern password algorithm (bcrypt or Argon2), encrypted data files and automatic daily backups to an external drive or cloud storage.
6. Export reports to Excel and PDF for the board, auditors and UMRA returns.
7. Introduce it in stages: train staff, enter existing members using the opening-balance option, and run the system alongside the paper books for one to two months before switching fully.